DeepSeek官方开源昇腾基础组件与昇腾共建高效易用的AI芯片软件生态

作者：邹楠、张雨晨、俞郑中、苗方正

时间：2026/09/30

https://gitcode.com/cann

DeepSeek × 昇腾：共建开源基础设施组件

面向大模型训练与推理场景，面向昇腾算力平台，推出核心算子库、分布式通信库

01 **TileLang & TileKernels**

基于 TileLang 封装昇腾底层指令，实现高性能训推 TileKernels 算子

02 **DeepEP**

MoE 专家并行通信库，兼顾高吞吐与低延迟

03 **DeepGemm**

FP8 GEMM 算子库，支持稠密与 MoE 分组矩阵乘

04 **FlashMLA**

MLA 解码注意力内核，针对变长序列优化

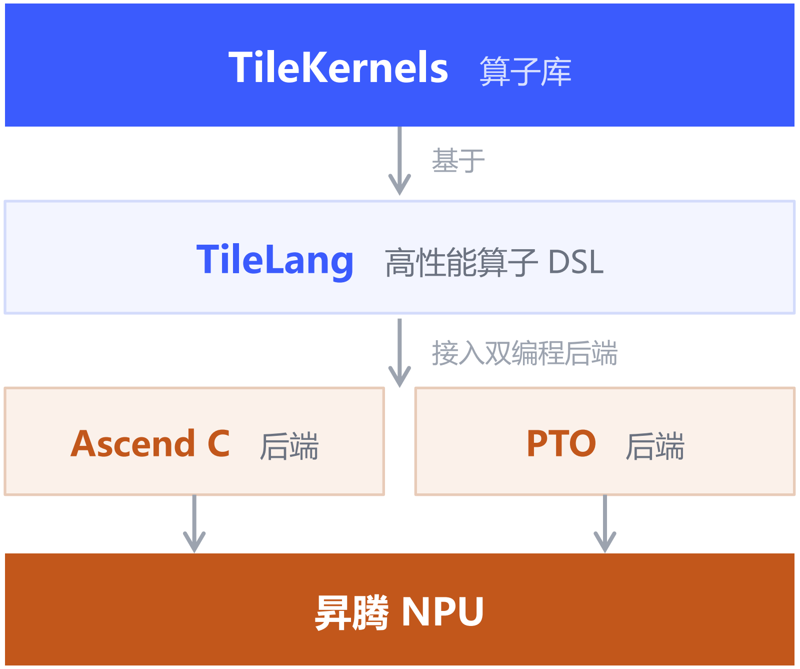
05 **DeepSelect**

Lightning Indexer TopK 选择与采样内核的高性能实现

06 **DeepJIT**

C++20 JIT 内核运行时，昇腾的编译、缓存与加载

TileKernels 编程路径


TileKernels 算子库
基于
TileLang 高性能算子 DSL
接入双编程后端
Ascend C 后端
PTO 后端
昇腾 NPU

一套 DSL 源码，经双后端编译，输出昇腾高性能算子

TileLang： 以 Tile 为单位编写高性能算子

Python DSL 算子前端 → **Tile / 数据流** 搬运、计算、写回 → **Pass** 内存管理、流水、同步 → **Ascend C / PTO** 生成可加载实现

TileLang 解决什么

① 把大张量拆成 Tile
以Tile为单位组织内存访问和计算，数据流更清晰。

② 保留性能控制入口
Tile 大小、并行域、流水和多缓冲仍然可以显式表达

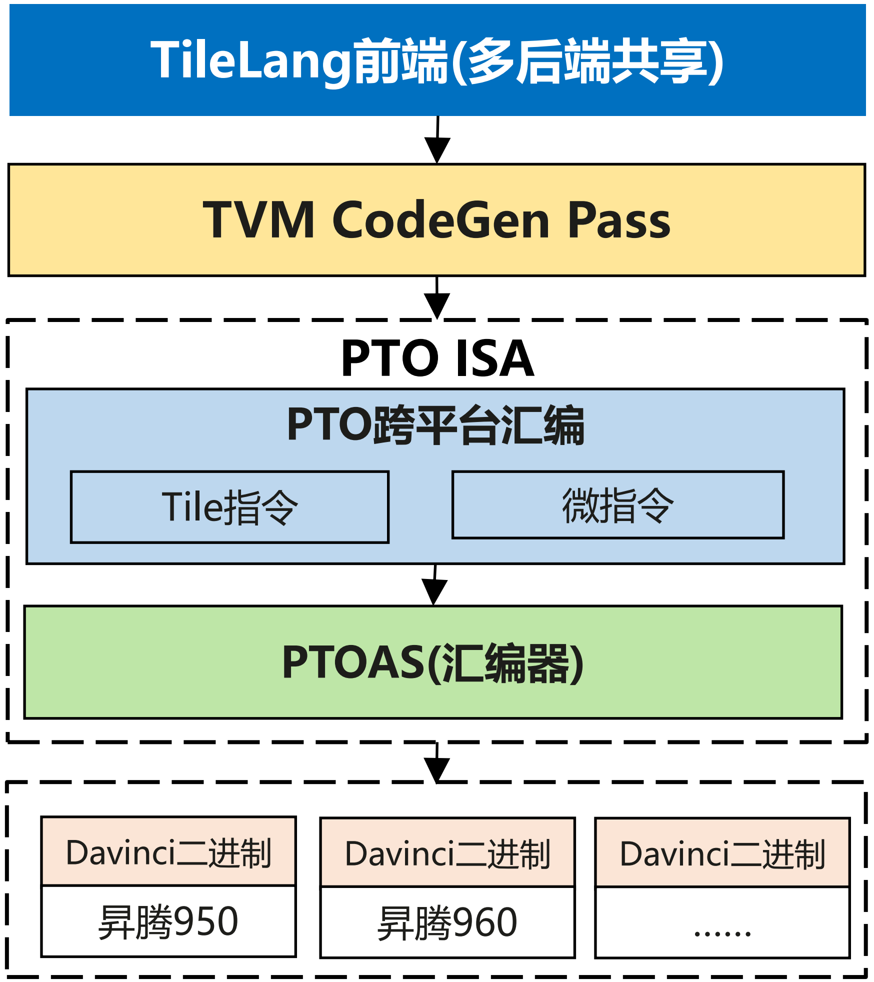
③ 复用公共编译能力
布局推断、边界处理、调度与同步不必在每个后端重复实现。

编写算子能达到什么效果

前端更简单	编程语法简洁，减少硬件依赖的编码，把注意力放在算子的核心逻辑与数据流。
融合更自然	框架负责内存管理和复用，cube和vector代码在前端无需显示区分 AIC 和 AIV。
pass层可复用	昇腾上的AscendC和PTO后端共用pass层，实现同一套前端代码灵活对接不同后端。

Tile 是搬运、计算和内存复用的基本单位

PTO ISA：共享TileLang前端表达，打造简单易用、极致性能、跨代兼容指令集系统

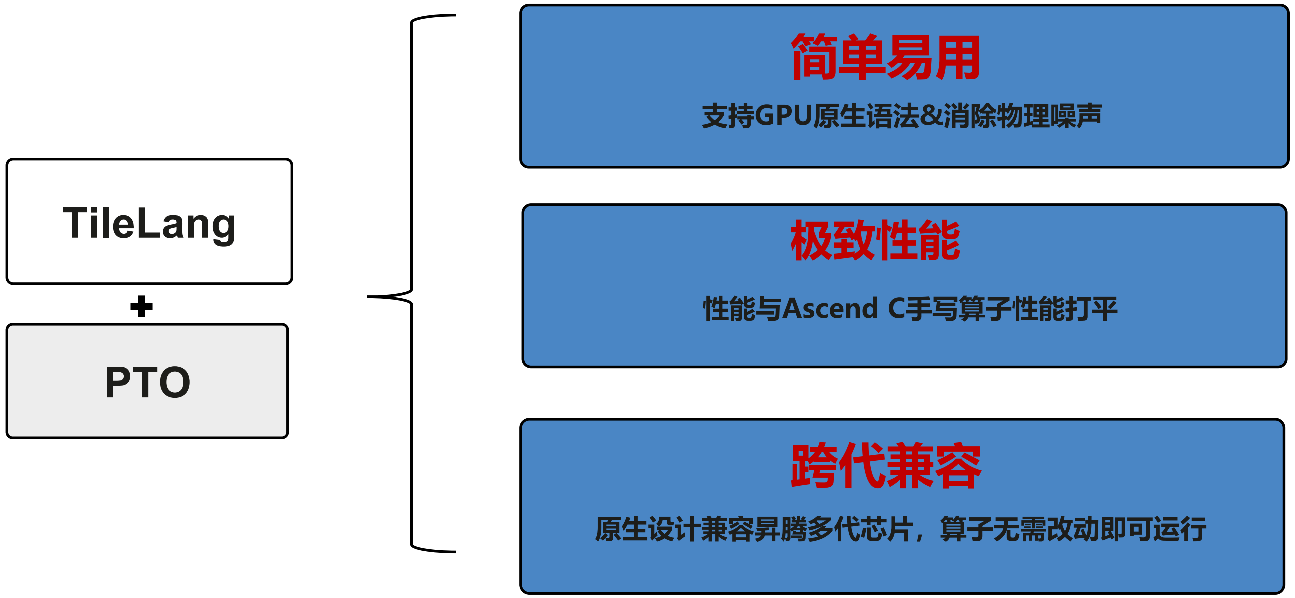

TileLang前端(多后端共享)
TVM CodeGen Pass
PTO ISA
PTO跨平台汇编
Tile指令
微指令
PTOAS(汇编器)
Davinci二进制
昇腾950
Davinci二进制
昇腾960
Davinci二进制
……

关键能力

PTO跨平台汇编：自定义跨平台Tile指令、微指令集标准

PTOAS：PTO汇编器工具链，编译优化&屏蔽不同代际芯片差异，使能指令集的高性能&跨平台

关键收益


TileLang
+
PTO

简单易用

支持GPU原生语法&消除物理噪声

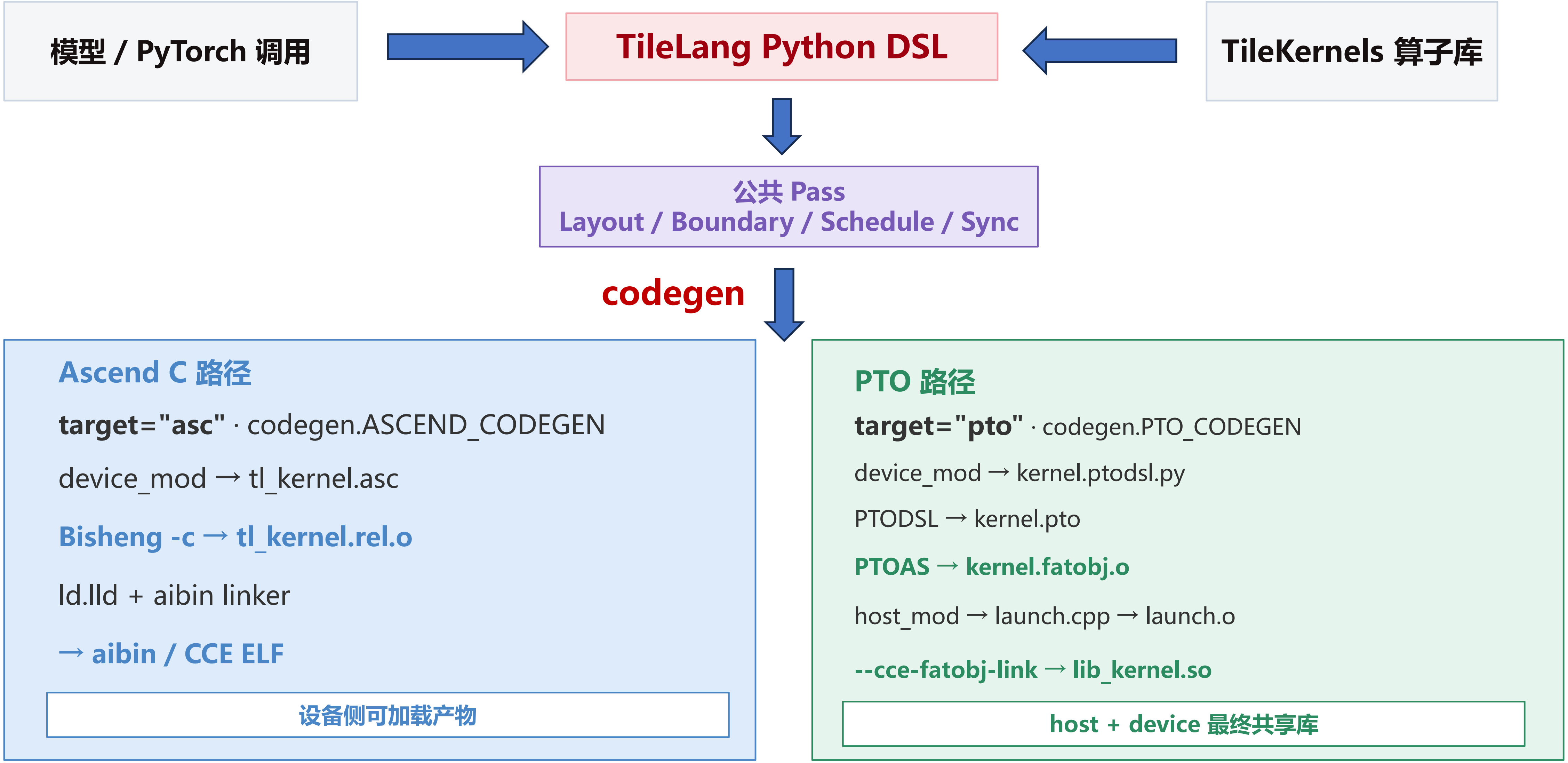
极致性能

性能与Ascend C手写算子性能打平

跨代兼容

原生设计兼容昇腾多代芯片，算子无需改动即可运行

同一份前端代码对接 AscendC & PTO



模型 / PyTorch 调用

TileLang Python DSL

TileKernels 算子库

公共 Pass
Layout / Boundary / Schedule / Sync

codegen

Ascend C 路径

target="asc" · codegen.ASCEND_CODEGEN

device_mod → tl_kernel.asc

Bisheng -c → tl_kernel.rel.o

ld.lld + aibin linker

→ aibin / CCE ELF

设备侧可加载产物

PTO 路径

target="pto" · codegen.PTO_CODEGEN

device_mod → kernel.ptodsl.py

PTODSL → kernel.pto

PTOAS → kernel.fatobj.o

host_mod → launch.cpp → launch.o

--cce-fatobj-link → lib_kernel.so

host + device 最终共享库

TileLang 编程范式

① 内存架构抽象

针对硬件内存层级设计的前端内存申请原语

UB -> alloc_shared

register -> alloc_fragment

L1 -> alloc_l1

L0 -> alloc_l0a/alloc_l0b/alloc_l0c

用 Tile 形状、存储 scope 和 buffer version 表达数据放在哪里；

② 算子编程原语

核任务持久化、向量化、自动流水线以及计算原语

T.Persistent

T.Parallel

T.Pipelined

T.SimtVF/T.SimdVF

T.copy / dual_copy

T.gemm

T.reduce

覆盖核启动、数据搬运、计算、调度、VF融合

③ 架构细节屏蔽

使用一系列pass屏蔽架构细节，简化前端算子编程逻辑

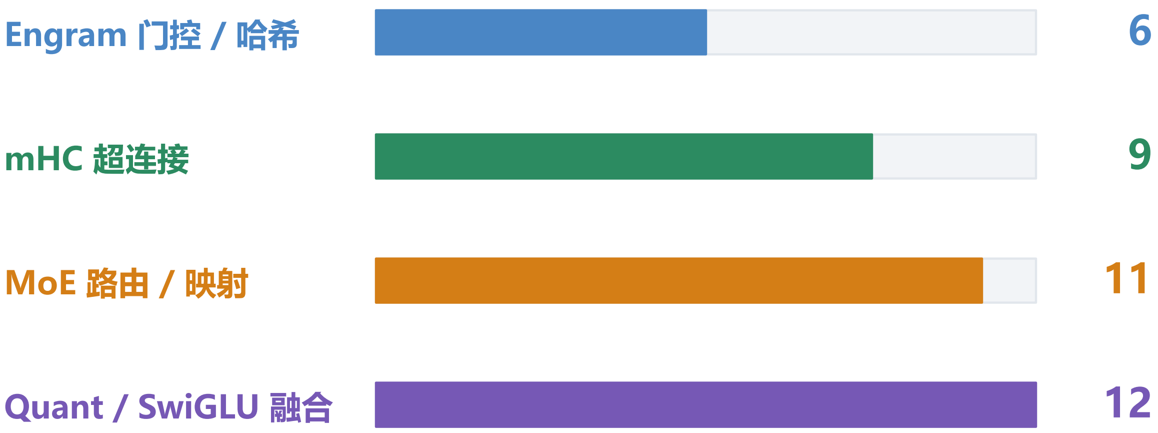
AutoSchedule: 估算延迟 → 分配核 → 规划多缓冲 → 插入同步

公共 Pass: 布局推断、边界处理、UB 合并、线程同步

前端保留 Tile 形状、作用域和数据依赖，专注于描述数据流

TileKernels：基于TileLang框架的算子库

40+



有效带宽 = 测试定义的 I/O 字节数 ÷ 执行耗时

mHC Post 前向

3.49 TB/s

耗时：302.72 μs

BF16；N=8192，H=7168，M=4

per-channel 量化

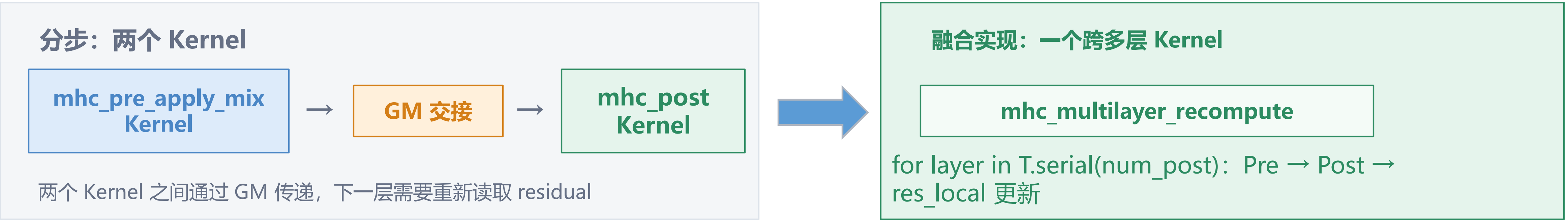
3.30 TB/s

耗时：53.05 μs

BF16；N=8064，H=7168

融合重计算：mHC Pre + Post

输入规模：batch=1，seq=8192，hidden=7168，mhc_mult=4；10 层，9 次 Post



分步：两个 Kernel

mhc_pre_apply_mix Kernel → GM 交接 → mhc_post Kernel

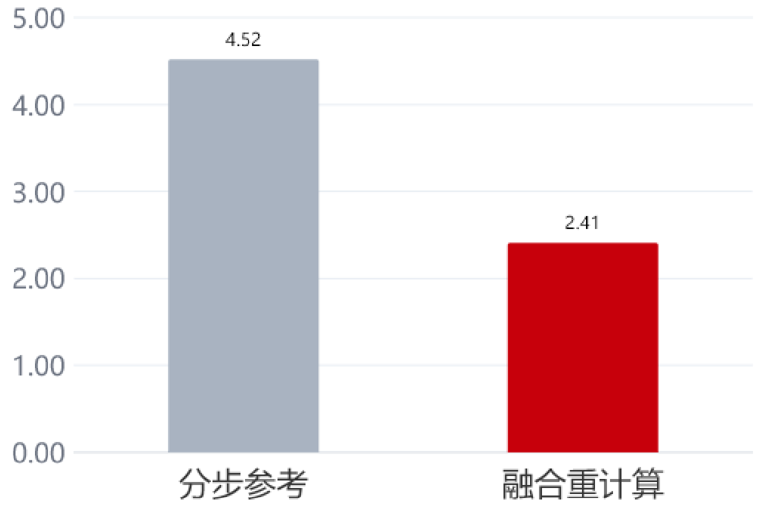
两个 Kernel 之间通过 GM 传递，下一层需要重新读取 residual

融合实现：一个跨多层 Kernel

mhc_multilayer_recompute

for layer in T.serial(num_post)：Pre → Post → res_local 更新

执行时间（ms）



mhc_pre_apply_mix + mhc_post：107行

mhc_multilayer_recompute：144行

1.88× **加速比**

融合的核心：一个跨层 Kernel + 片上状态 + 双缓冲预取

CANN 社区的学习与开发资源

开发课程

cann-learning-hub

TileLang 算子开发课程

编程基础、精度验证与优化

算子库

ops-tilelang

面向昇腾的算子实现

参考、复用与持续优化

Agent 辅助

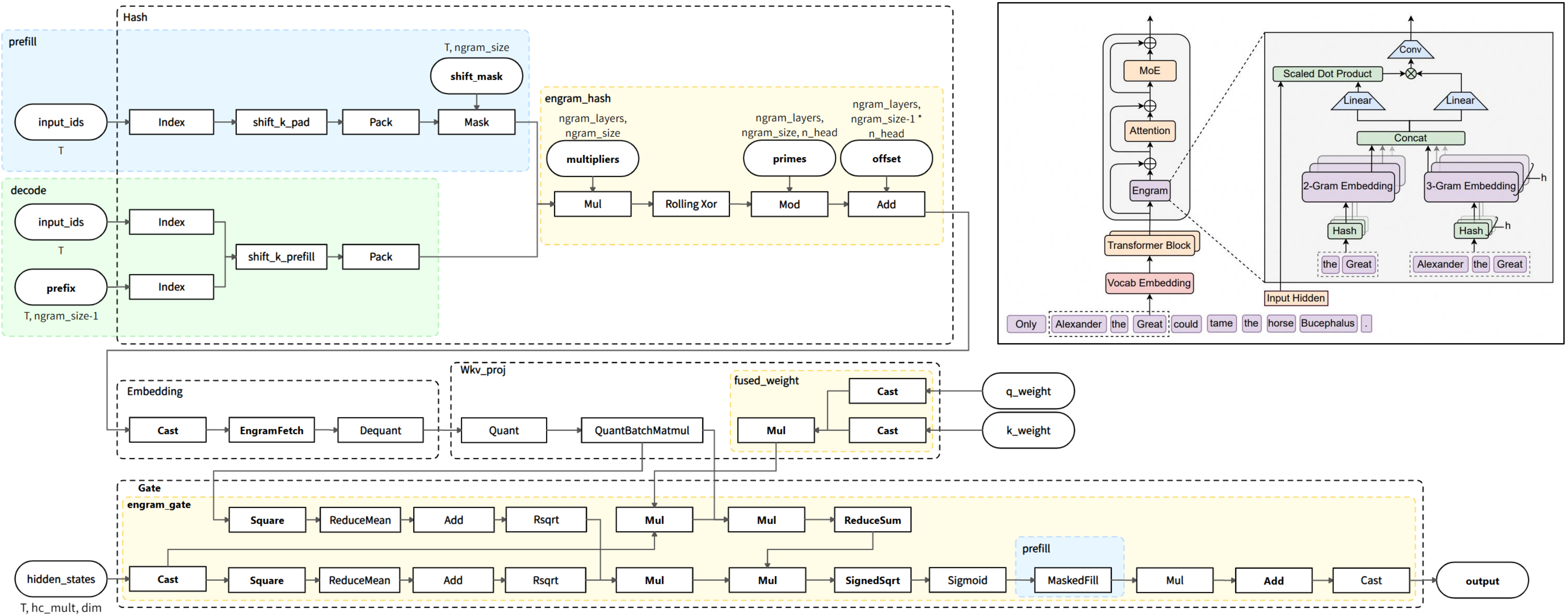
cannbot-tilelang

辅助算子生成与调优

方案设计、代码生成与验证

课程入门
算子实践
Agent 辅助开发

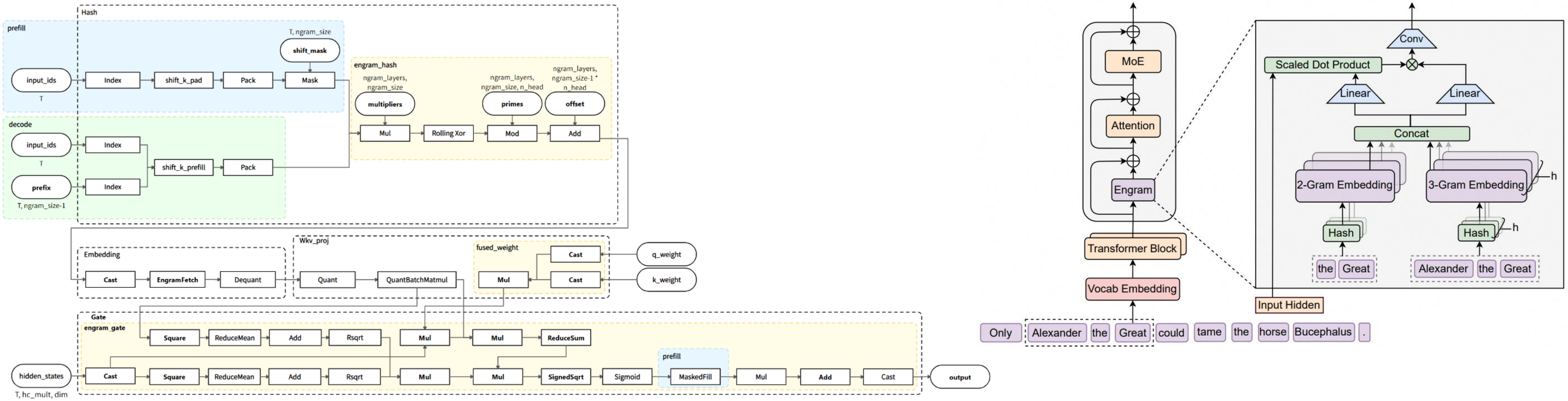
Deepseek V4.1 Engram与TileKernels


Hash
prefill
T, ngram_size
shift_mask
input_ids
T
Index
shift_k_pad
Pack
Mask
decode
input_ids
T
Index
shift_k_prefill
Pack
prefix
T, ngram_size-1
Index
engram_hash
ngram_layers, ngram_size
multipliers
ngram_layers, ngram_size, n_head
primes
ngram_layers, ngram_size-1 * n_head
offset
Mul
Rolling Xor
Mod
Add
Embedding
Cast
EngramFetch
Dequant
Wkv_proj
Quant
QuantBatchMatmul
fused_weight
Cast
Mul
Cast
q_weight
k_weight
Gate
engram_gate
Square
ReduceMean
Add
Rsqrt
Mul
Mul
ReduceSum
hidden_states
T, hc_mult, dim
Cast
Square
ReduceMean
Add
Rsqrt
Mul
Mul
SignedSqrt
Sigmoid
prefill
MaskedFill
Mul
Add
Cast
output
MoE
Attention
Engram
Transformer Block
Vocab Embedding
Only
Alexander
the
Great
could
tame
the
horse
Bucephalus
.
Conv
Scaled Dot Product
Linear
Linear
Concat
2-Gram Embedding
3-Gram Embedding
h
Hash
Hash
h
the
Great
Alexander
the
Great
Input Hidden

Engram 是解耦「记忆」与「计算」的条件记忆模块，以可训练的 n-gram 哈希记忆补充固定上下文之外的语言先验

https://gitcode.com/cann

Deepseek V4.1 Engram与TileKernels

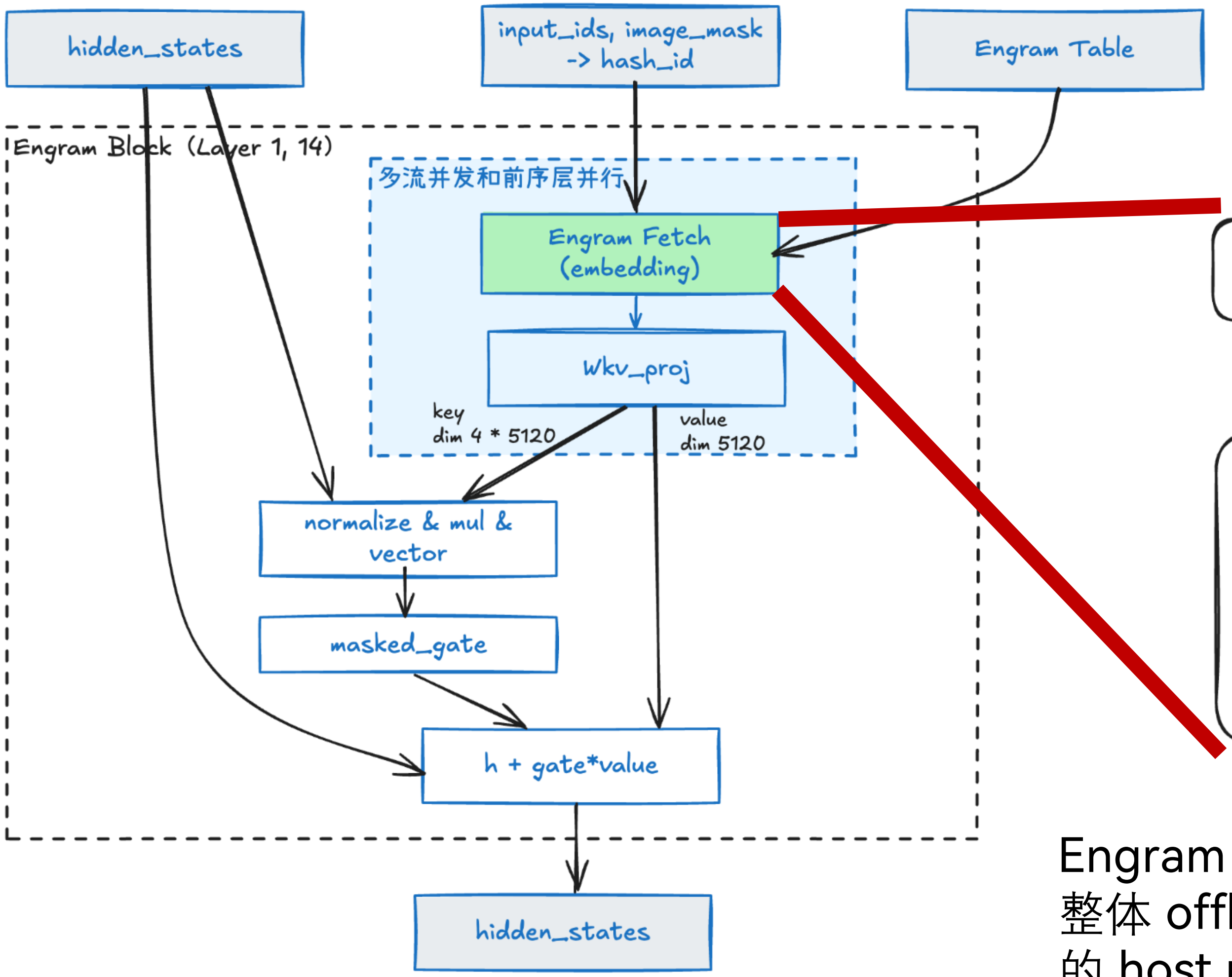

Hash
prefill
input_ids
T
Index
shift_k_pad
Pack
Mask
T, ngram_size
shift_mask
decode
input_ids
T
prefix
T, ngram_size-1
Index
Index
shift_k_prefill
Pack
engram_hash
ngram_layers, ngram_size
multipliers
ngram_layers, ngram_size, n_head
primes
ngram_layers, ngram_size-1 * n_head
offset
Mul
Rolling Xor
Mod
Add
Embedding
Cast
EngramFetch
Dequant
Wkv_proj
Quant
QuantBatchMatmul
fused_weight
Cast
Cast
Mul
q_weight
k_weight
Gate
engram_gate
Square
ReduceMean
Add
Rsqrt
Mul
Mul
ReduceSum
hidden_states
T, hc_mult, dim
Cast
Square
ReduceMean
Add
Rsqrt
Mul
Mul
SignedSqrt
Sigmoid
prefill
MaskedFill
Mul
Add
Cast
output
MoE
Attention
Engram
Transformer Block
Vocab Embedding
Only
Alexander
the
Great
could
tame
the
horse
Bucephalus
.
Conv
Scaled Dot Product
Linear
Linear
Concat
2-Gram Embedding
3-Gram Embedding
h
Hash
Hash
h
the
Great
Alexander
the
Great
Input Hidden

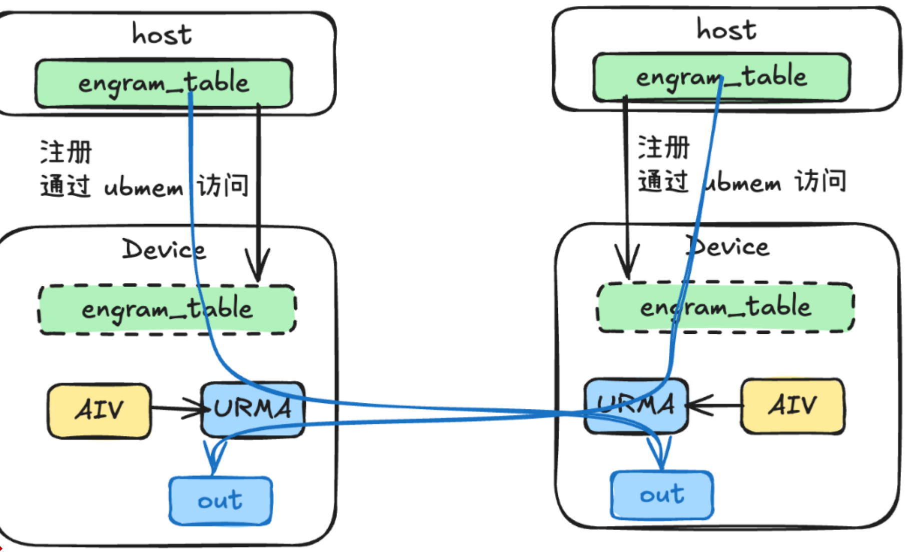
Engram TileLang 性能对比

Prefill 融合算子性能对比（单位 μs，单次调用）：

Prefill tokens	engram_hash TileLang	engram_hash PyTorch	fused_weight TileLang	fused_weight PyTorch	engram_gate_fwd TileLang	engram_gate_fwd PyTorch
8K	25.877	1118.970	2.856	542.804	355.302	4129.162
16K	43.668	1989.527	3.202	1073.346	724.430	8112.683

Engram --- Fetch

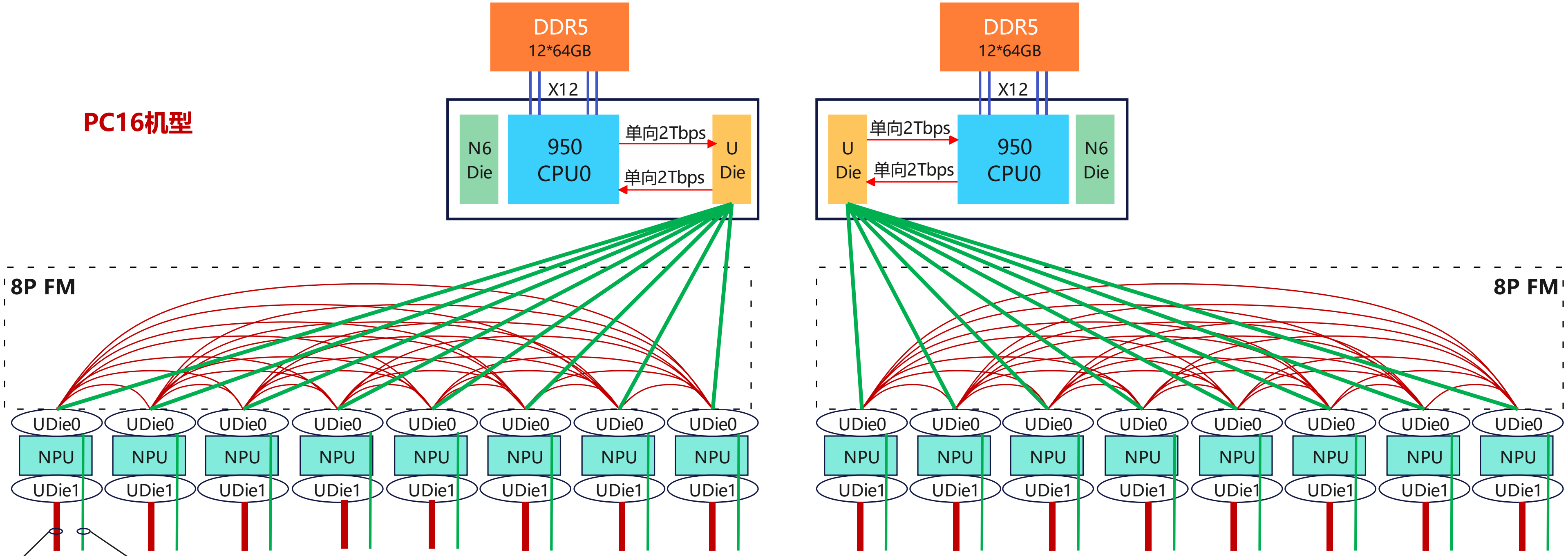

hidden_states
input_ids, image_mask
-> hash_id
Engram Table
Engram Block (Layer 1, 14)
多流并发和前序层并行
Engram Fetch
(embedding)
Wkv_proj
key
dim 4 * 5120
value
dim 5120
normalize & mul &
vector
masked_gate
h + gate*value
hidden_states


host
engram_table
注册
通过 ubmem 访问
Device
engram_table
AIV
URMA
out
host
engram_table
注册
通过 ubmem 访问
Device
engram_table
URMA
AIV
out

Engram 表本体两表合计 197 GB，采用多级多卡切分，并整体 offload 到 host 内存上存储，通过 UB 能力从分布式的 host memory 中 fetch Engram

Atlas 950 SuperPoD Flex

PC16机型


DDR5
12*64GB
X12
N6
Die
950
CPU0
单向2Tbps
单向2Tbps
U
Die
DDR5
12*64GB
X12
U
Die
单向2Tbps
单向2Tbps
950
CPU0
N6
Die
8P FM
8P FM
UDie0
NPU
UDie1

D2D出框带宽
8*UB X4 850GB/s （双向）
（8*106.25GB/s）

SO网络 UB 协议
1*400Gbps

NPU UB port map

Port	0	1	2	3	4	5	6	7	8
UDie1		8 口 for 128 超节点 (scale-up)							
Port	1口H2D	7 口 8 卡 FM							Scale-out

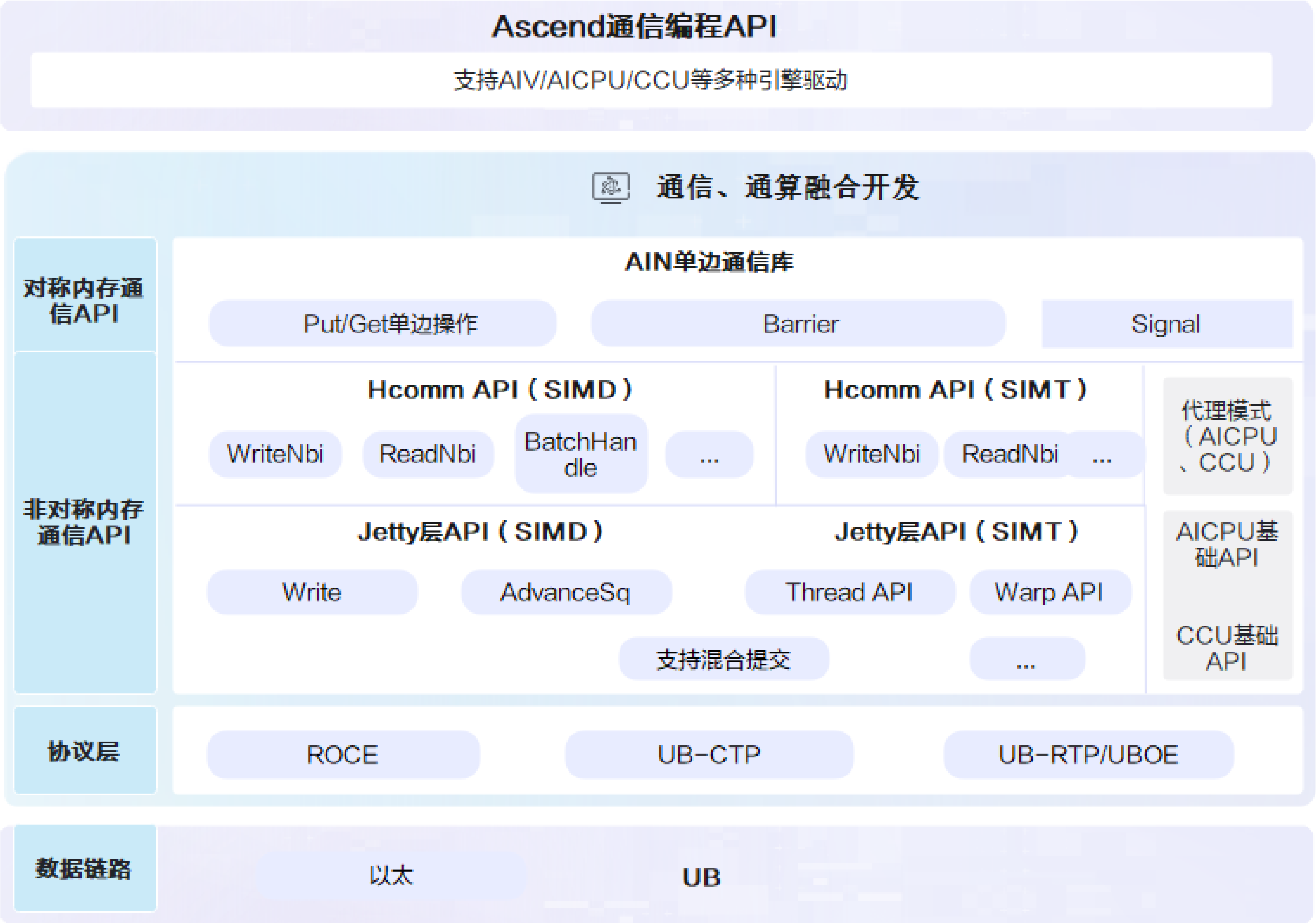
CPU UB port map

Port	0	1	2	3	4	5	6	7	8
UDie	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	
	NPU1	NPU2	NPU3	NPU4	NPU5	NPU6	NPU7	NPU8	

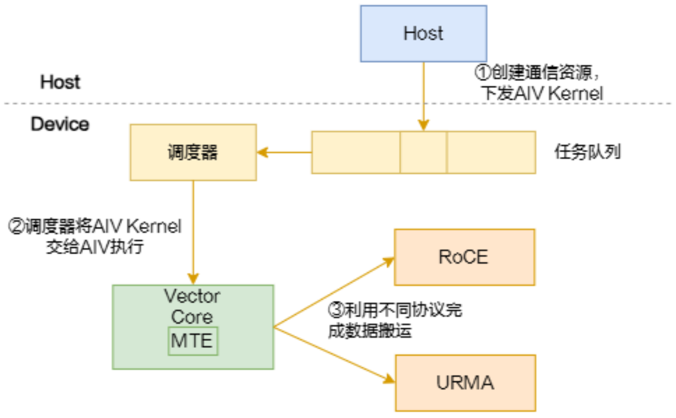
SuperPoD Flex 机型和 UBL128 组网方案

SuperPoD Flex可实现128卡3.2Tbps单层交换Scale-Up网络和256K卡两层Scale-Out网络，可以支持超低时延推理、和前沿基座模型的大模型训练需求。

ASC-COMM: 自定义通信编程API



Ascend通信编程API
支持AIV/AICPU/CCU等多种引擎驱动
通信、通算融合开发
对称内存通信API
非对称内存通信API
协议层
数据链路
AIN单边通信库
Put/Get单边操作
Barrier
Signal
Hcomm API（SIMD）
WriteNbi
ReadNbi
BatchHandle
...
Hcomm API（SIMT）
WriteNbi
ReadNbi
...
代理模式（AICPU、CCU）
JettyAPI（SIMD）
Write
AdvanceSq
JettyAPI（SIMT）
Thread API
Warp API
支持混合提交
...
AICPU基础API
CCU基础API
ROCE
UB-CTP
UB-RTP/UBOE
以太
UB



Host
Host
Device
①创建通信资源，下发AIV Kernel
调度器
任务队列
②调度器将AIV Kernel交给AIV执行
Vector Core
MTE
③利用不同协议完成数据搬运
RoCE
URMA

AIV直驱模式

- **Ain单边通信API**

基于对称内存的单边通信库。AIN通过目标成员、窗口和偏移发起Put、Get或Signal等任务，简化集群编程

- **Hcomm层API**

HCOMM控制点对点任务通过Channel确定通信对端，并由本地地址和远端地址分别指定数据的来源和目标。该模型适合目的地址动态变化、Buffer布局不一致以及需要精细的场景。

- **Jetty层API**

Jetty层提供底层URMA通信原语，提供最灵活、高性能的通信控制。

- **总结**

当前通信编程API同时支持SIMD和SIMT编程调用。SIMT优势在于并发优势。能够使用较少的计算资源进行批量下发读写请求

DeepEP-Ascend

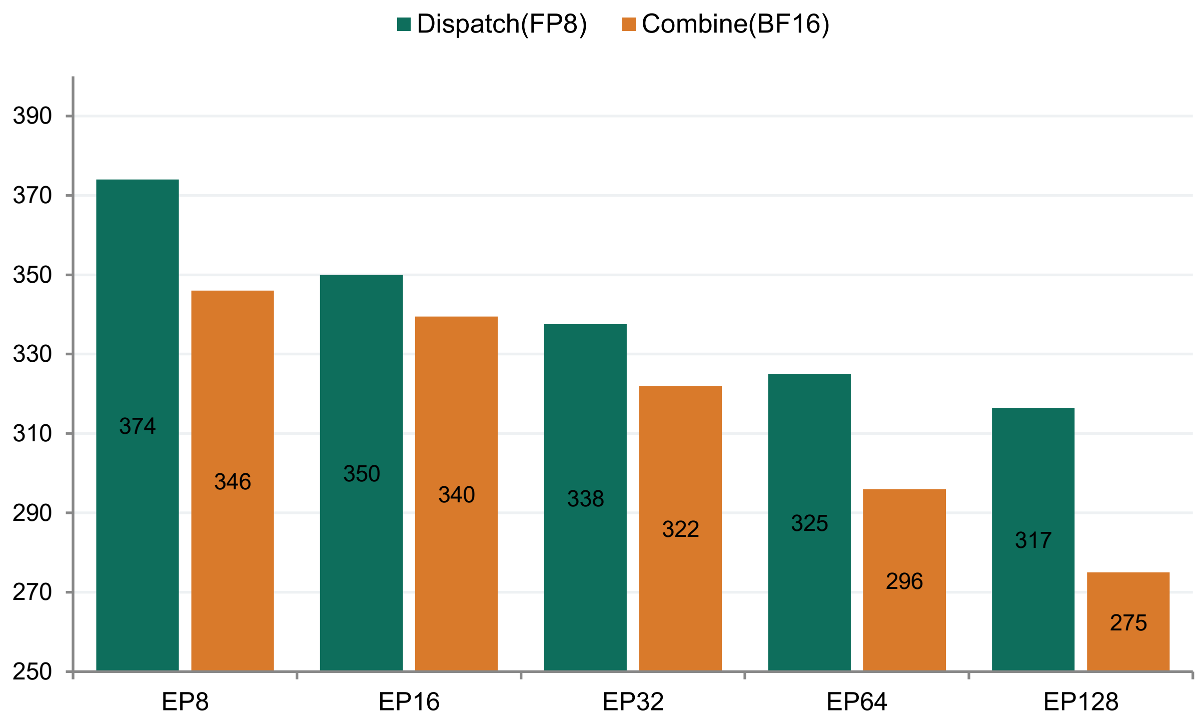
DeepSeek开源的面向昇腾 NPU 的并行通信库

EPBuffer:MoE dispatch / combine,BF16 / FP8,缓存句柄与延迟 epilogue

BucketBuffer: CP / DP 批量 all-gather

PPBuffer： PP并行的send、recv

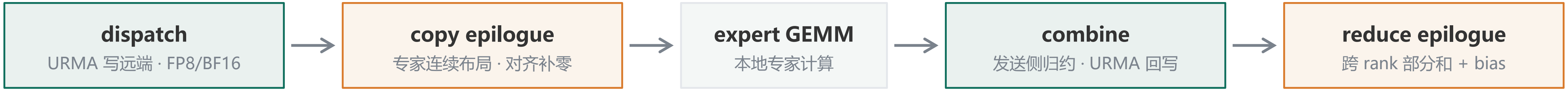
EngramBuffer: 支持Engram Table offload, 远端host 内存的 Embedding 表 Fetch



通信带宽达硬件理论带宽90%以上

DeepEP-Ascend

EPBuffer


dispatch
URMA 写远端 · FP8/BF16
copy epilogue
专家连续布局 · 对齐补零
expert GEMM
本地专家计算
combine
发送侧归约 · URMA 回写
reduce epilogue
跨 rank 部分和 + bias

Dispatch(分发,三阶段)

① **路由规划**:每 AIV 一个 512 线程 SIMT 常驻 worker,单趟完成直方图、ballot 按 rank 去重、原子加分配序号;跨核前缀和用 warp inclusive sum,4K token 约 6 μs。

② **URMA 传输**:同目的 rank 的多 token 合并成一条 SQE(48B 头 + 6 对 SGE),warp 内 lane 分工直写;本地 token 走 GM→UB→GM 流水;写完一次 doorbell。

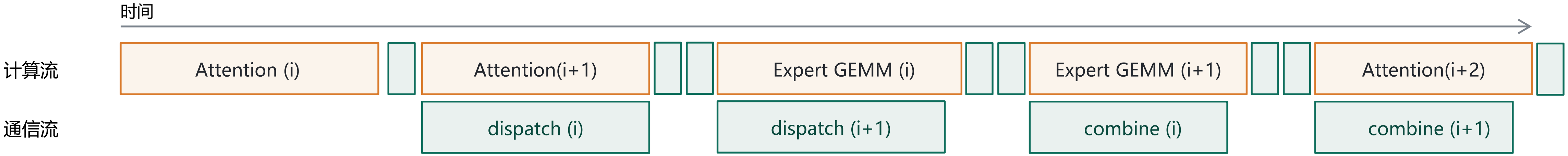
③ **copy epilogue**:SIMD 频率直方图 + "原子广播"单趟算出专家写指针,流式展开为 128 对齐布局并补零,顺带产出 combine 元数据。

Combine(归约,两阶段)

① **发送侧归约**:同一源 token 的 top-k 份专家输出先经 RingReducer 以 FP32 累加合并,只回写一份部分和——网络流量最多降为 1 / topk。

② **reduce epilogue**:接收侧按 bitset 给每个来源 rank 保留一份部分和,与 bias 一起 FP32 累加;

通信-计算流水掩盖(DualPipe 式,训练场景)


时间
计算流
通信流
Attention (i)
Attention(i+1)
Expert GEMM (i)
Expert GEMM (i+1)
Attention(i+2)
dispatch (i)
dispatch (i+1)
combine (i)
combine (i+1)

epilogue 延迟到 wait() 才执行;参照 DeepEP / DualPipe 双向流水,把另一微批次的 Attention 与共享专家 FFN 填入 dispatch / combine 的通信窗口

详细的实现代码在 DeepSeek的DeepEP-Ascend仓库，感兴趣的同学可以直接去看。

DeepGEMM-Ascend

DeepSeek开源的面向昇腾 NPU 的高性能GEMM算子库。五类核心算子贴着硬件瓶颈设计与优化

① **Dense GEMM** **BF16 / FP8 / FP4**	**瓶颈：MAD 单元利用率**。五级流水 GM→L1(nd2nz 转 NZ 分形)→L0A/B→L0C→GM，加载、乘加、写回三条流水重叠；FP8/FP4 缩放因子按 32 元素一组随 K 块加载；set/wait精确配对替代全局屏障；无后处理时 L0C 直写 GM（DirectStore）。
② **Grouped GEMM** **MoE 专家分组**	**瓶颈：分组调度开销**。token 按专家连续排布，统一调度器按组切换权重与缩放因子；host 侧启发式按形状自动选分块、流水级数与核数，代价模型与内核强制同步演进；4–8 组实测与稠密 GEMM 算力持平。
③ **MQA Logits** **Lightning Indexer**	**瓶颈：FIX 写回管道（99% 饱和）**。优化全部围绕写回展开：prefill/decode 双调度器，分页 KV 元数据 host 侧预生成；SIMD 把缩放因子打包成 16-token 片段；非对齐行头经 UB 暂存对齐后再写 GM。
④ **MegaMoE** **EP 融合算子**	**瓶颈：专家GEMM计算瓶颈**。dispatch + 双分组 GEMM + SwiGLU + combine 融合为单内核；ubmem 内存直达访问；通信与计算重叠。
⑤ **HC Prenorm GEMM** **mHC 超连接**	**瓶颈：HBM 写带宽**。1C2V 混合核：向量核一次扫描完成 BF16→FP32 转换与行平方和，Cube 核复用常驻 B 做 TF32 乘加；类型转换、归约、GEMM、写回全部留在单内核，K 切分核内原子归约。

DeepGEMM-Ascend

性能测试

算子	实测关键指标（Ascend 950DT，冷 L2）	极限逼近度
Dense GEMM	4096×7168×16384：BF16 431 / FP8 861 / FP4 1701 TFLOPS	**98.3%–99.8% 硬件极限**
M-Grouped GEMM	4–8 组、单组 4096–8192 token：818–862 TFLOPS	**与稠密持平**
MQA Logits	Decode FP4：708 TFLOPS；FP8 访存 2006 GB/s	**FIX 管道 99% 饱和**
MegaMoE	E=384、16384 token：831–846 TFLOPS，通信 135 GB/s	**通信被计算掩盖**
HC Prenorm GEMM	M=8192、K=28672：3463 GB/s	**逼近 HBM 写带宽**

· 以轻量代码量在昇腾 950 上复现 DeepGEMM 全部算子族，API 完全兼容，稠密 GEMM 达硬件极限的 98.3%–99.8%。

· 性能来自：NZ 分形 + 多级流水、按形状的 JIT 配置选择、面向硬件瓶颈（MAD / FIX 管道 / HBM 写带宽）的算子级融合。

CANN

详细的实现代码在 DeepSeek的DeepGEMM-Ascend仓库，感兴趣的同学可以直接去看。

Deepseek V4.1 网络结构与融合算子应用

DSV4.1-Flash

结构升级〔CED〕

- **CED 结构**：Prefill 激活降至约 8B、Decode 约 16B
- Backbone 约 552B；1 共享 + 384 路由专家，topK-6

存储优化〔KV Cache〕

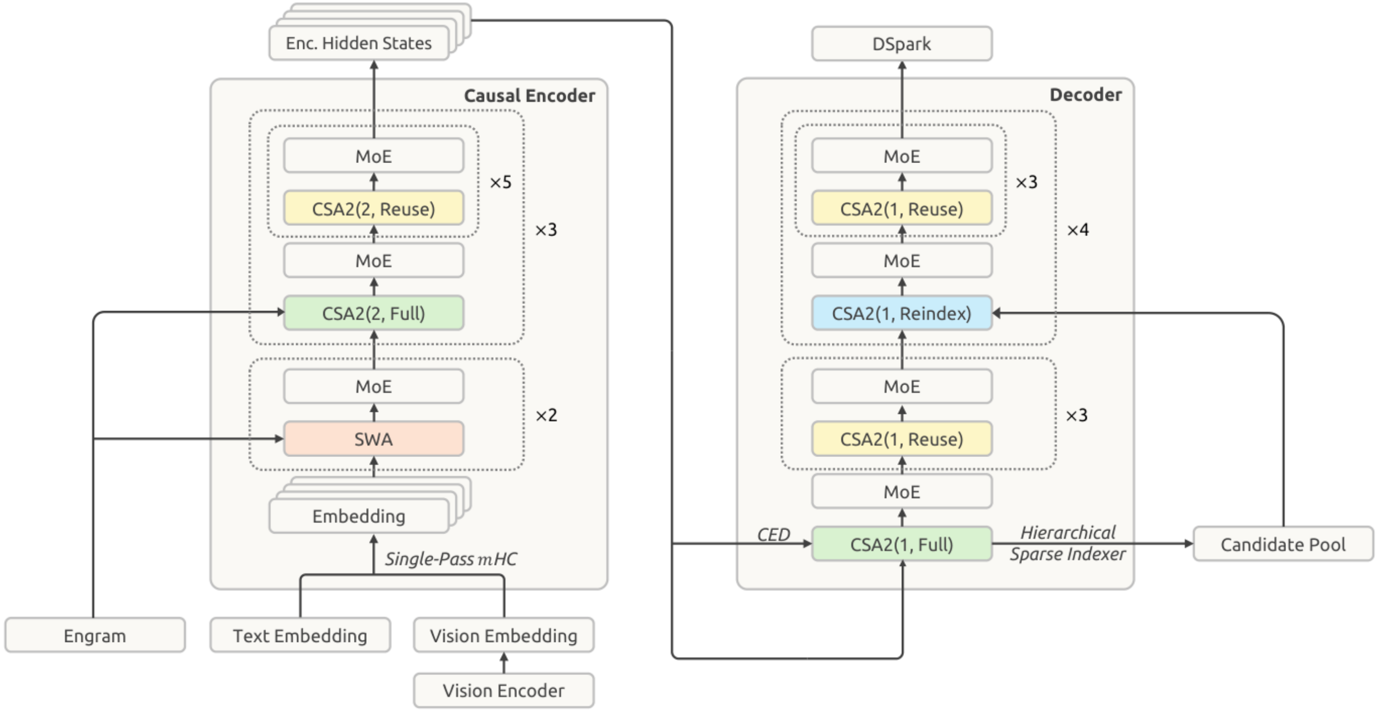
- **CSA2 + SharedKV + FP4 量化**，**2× + 1×** 分层压缩
- 全局 KV Cache 约 **890 B/token** ≈ V4-Flash 的 **1/4**
- **Bounded Replay**：持久化 KV 再降至约 **1/8**
- **压缩机制转变**：取消纯 HCA 压缩，移除 indexer-compressor
  高倍率压缩 → **低倍率 + 稀疏 + 共享 cache**

算力优化〔Attention〕

- **固定 TopK + SWA**，限制参与范围、降低 Attention FLOPs

能力扩展〔多模态〕

- **统一多模态**：文本 / 图像 多模态统一Mask，保持 1M 上下文


Enc. Hidden States
Causal Encoder
MoE
CSA2(2, Reuse)
×5
×3
MoE
CSA2(2, Full)
MoE
SWA
×2
Embedding
Single-Pass mHC
Engram
Text Embedding
Vision Embedding
Vision Encoder
DSpark
Decoder
MoE
CSA2(1, Reuse)
×3
×4
MoE
CSA2(1, Reindex)
MoE
CSA2(1, Reuse)
×3
MoE
CED
CSA2(1, Full)
Hierarchical
Sparse Indexer
Candidate Pool

Deepseek V4.1 网络结构与融合算子应用

DSV4.1-Flash

结构升级〔CED〕

- **CED 结构**：Prefill 激活降至约 8B、Decode 约 16B
- Backbone 约 552B；1 共享 + 384 路由专家，topK-6

存储优化〔KV Cache〕

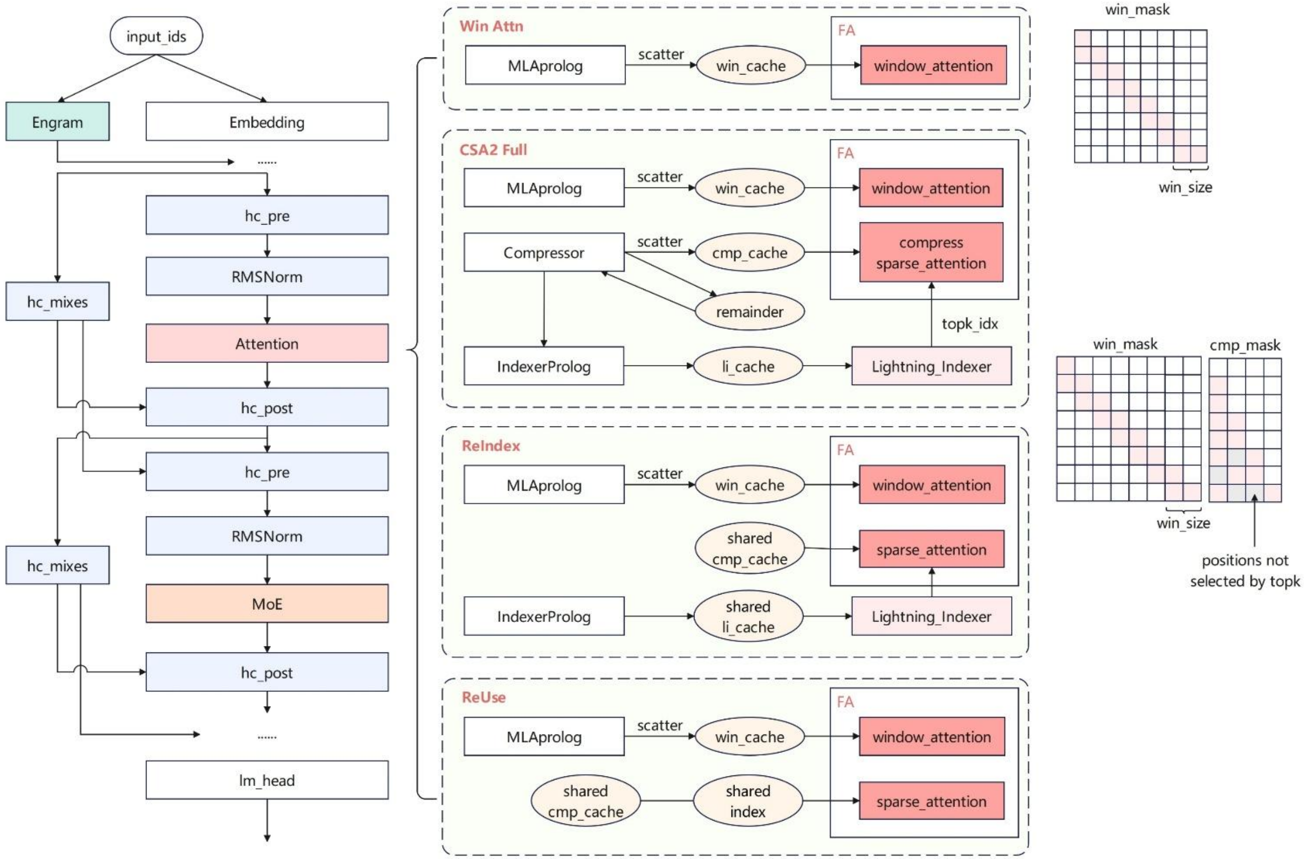
- **CSA2 + SharedKV + FP4 量化**，**2× + 1×** 分层压缩
- 全局 KV Cache 约 **890 B/token** ≈ V4-Flash 的 **1/4**
- **Bounded Replay**：持久化 KV 再降至约 **1/8**
- **压缩机制转变**：取消纯 HCA 压缩，移除 indexer-compressor
  高倍率压缩 → **低倍率 + 稀疏 + 共享 cache**

算力优化〔Attention〕

- **固定 TopK + SWA**，限制参与范围、降低 Attention FLOPs

能力扩展〔多模态〕

- **统一多模态**：文本 / 图像 多模态统一Mask，保持 1M 上下文


input_ids
Engram
Embedding
......
hc_pre
RMSNorm
Attention
hc_post
hc_pre
RMSNorm
MoE
hc_post
......
lm_head
hc_mixes
hc_mixes
Win Attn
MLAprolog
scatter
win_cache
FA
window_attention
CSA2 Full
MLAprolog
scatter
win_cache
FA
window_attention
Compressor
scatter
cmp_cache
compress sparse_attention
remainder
topk_idx
IndexerProlog
li_cache
Lightning_Indexer
ReIndex
MLAprolog
scatter
win_cache
FA
window_attention
shared cmp_cache
sparse_attention
IndexerProlog
shared li_cache
Lightning_Indexer
ReUse
MLAprolog
scatter
win_cache
FA
window_attention
shared cmp_cache
shared index
sparse_attention
win_mask
win_size
win_mask
cmp_mask
win_size
positions not selected by topk

FlashMLA 稀疏注意力，性能逼近理论峰值

Ascend Sparse Attention Forward · Prefill / Decoding

DeepSeek V4.1 模型的典型工况——DSA 稀疏注意力架构，每个 q token 只关注 topk 个最重要的 kv token，保证模型效证模型效果的同时大幅节约算力。

---

410 TFLOPS

Prefill 峰值算力

360 TFLOPS

Decoding峰值算力

来源：deepseek-ai/FlashMLA（2026-09-30 开源）· 华为昇腾 Ascend 950

FlashMLA 算子设计&优化技巧

gather 稀疏取 KV → Q · K^T 打分 → online softmax → S · V 累加 → Attention 输出

每个 AI Core 64 个 q head　　1C2V 协同计算

CUBE

QK[i+1] 与 SV[i] 交错执行，消除矩阵计算bubble

把 0 += S[i] · V[i] 延后一块，为 softmax 留出约 1024 cycle计算时间。

VECTOR ×2

索引预取(i+5) ➜ ND2NZ(i+3) ➜ softmax(i+1) ➜ 0 scale(i)

skip-scale 信号经 SS buffer + 跨核 flag，在 SIMD VF 与 CUBE 之间传递。

gather2　burst_count=2，一条 MTE2 搬两个 token，缓解发射队列深度 16 限制

高速 ND2NZ　vsstb 按 32B 块写回并 插入pad 消减 bank conflict，打满 UB 带宽

iCache miss　VLOOP 硬件循环替代软件展开、融合主尾循环，控制代码体积

KV 布局重排　token 的 data 与 scale 相邻放置，只需一次拷贝

skip-scale　少拷贝、少计算、降功耗

L1 bank 配平　张量对半拆进两个 bank，尽可能打高 L1 读写带宽

更加详细的技术报告和代码就在DeepSeek的FlashMLA仓库，感兴趣的同学可以直接去看。

DeepSelect

功能介绍

BF16 输入 [B, N]，返回 INT32 索引 [B, K]，K ≤ 4096
支持变长输入、索引升序与可选值输出

核心算法：在线过滤，按需重选

01 分段乱序

每页 2048 个元素
页间置换，页内连续

02 预热建阈值

预热 2–4 页求 TopK
建立初始阈值 τ

03 扫描过滤

遍历其余页
收集 value > τ 的候选

04 候选重选

达到触发量或扫描结束
合并重选并更新阈值

关键技术

按行分核

一行一个 Vector block
行内 SIMD 压缩候选

两级直方图

按高、低字节定位阈值
高字节计数累计并截断

三级缓冲

每批处理 16K 元素
协调数据搬运与计算

UB 空间复用

248 KiB 预算内复用
预热、候选与排序空间

https://gitcode.com/cann

CANN

DeepJIT：一次编译，全局共享

compile() 先计算缓存Key、再查两级缓存，命中即完全跳过编译

01

缓存KEY

包括源码、include 树、编译器版本与编译选项，任何改动将触发重编

→

02

内存缓存

进程内命中直接返回已加载 kernel，零编译加载开销

→

03

磁盘缓存

跨进程、跨节点复用已编译产物

→

04

编译并发布

未命中则触发编译；fsync 后原子 rename，达成并发安全

- **零成本嵌入**

  Header-only、无需链接库；

- **懒初始化**

  首次使用才探测Device与编译工具链，进程冷启动零负担，多卡环境按需构造

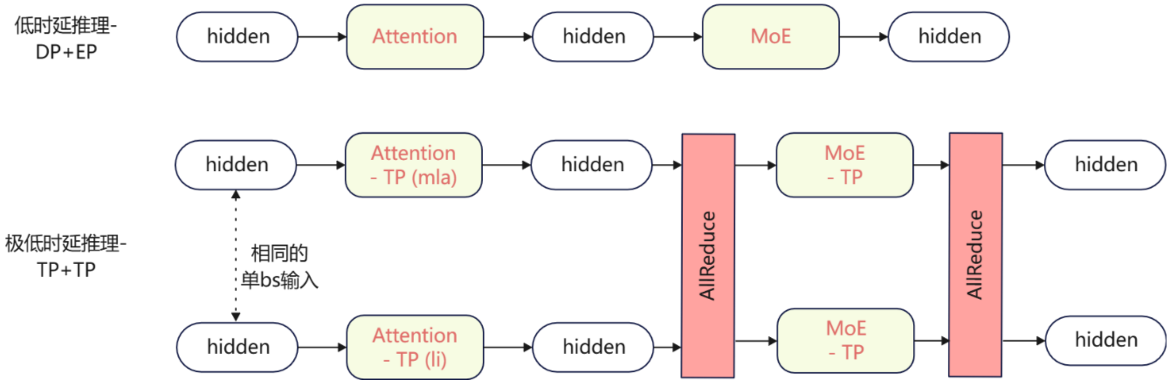
- **诊断开箱即用**

  昇腾汇编一键导出，编译命令、加载耗时、寄存器 spill 均可检查

```
auto kernel =jit->compile("scale", src);
// 首次编译，此后进程内外直接命中缓存
jit->launch(kernel, {.grid_dim = g, .block_dim = b}, output, input, count);
```

CANN

Deepseek V4.1 昇腾实践：推理与部署


低时延推理-DP+EP
hidden
Attention
hidden
MoE
hidden
极低时延推理-TP+TP
hidden
Attention - TP (mla)
hidden
AllReduce
MoE - TP
AllReduce
hidden
相同的单bs输入
hidden
Attention - TP (li)
hidden
MoE - TP
hidden

超节点高速互联，助力大EP低时延高吞吐

DP+EP部署策略，V41结构叠加高性能算子消减卡间负载失衡，支撑高吞吐部署

Global Batch Size	Chips	并行方式	数据类型	序列长度	TPOT (ms)	Throughput (Tokens/s)
384	32	DP+EP	Hybrid MXFP8-MXFP4	128K	4.40	2727
1536	32	DP+EP	Hybrid MXFP8-MXFP4	128K	9.41	5102
2560	32	DP+EP	Hybrid MXFP8-MXFP4	128K	13.79	5800

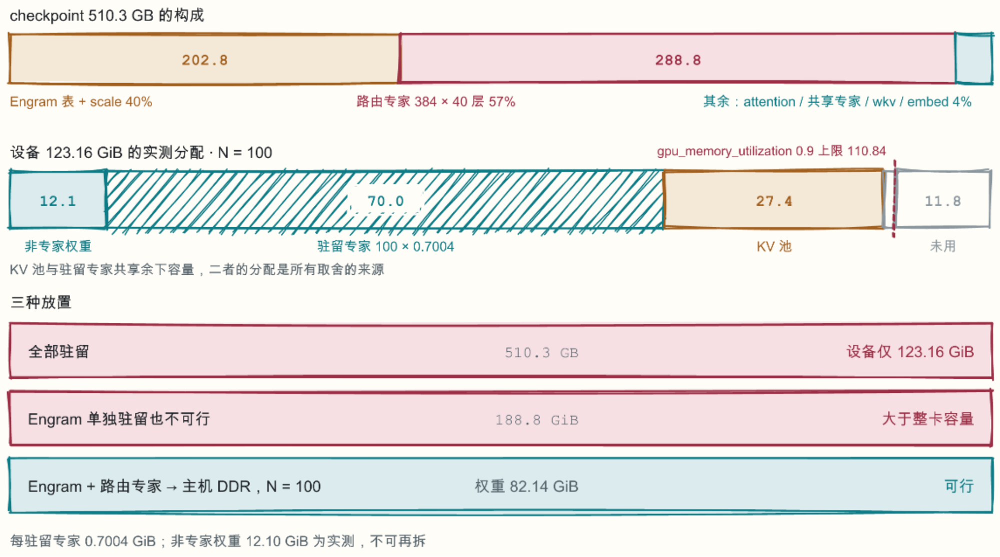
异构部署+TP切分，支撑极低时延硬件极限

基于模型结构自带的prolog/li/compressor多条链路，将NPU拆分为异构Worker，部分执行 MLA、部分执行 Indexer

Global Batch Size	Chips	DSpark	DataType	Seq Length	TPOT (ms)
1	8	7	Hybrid MXFP8-MXFP4	8192	2.x

注：性能数据基于DSpark7采集，平均步长5.7，用户可按照数据集实际接受率自行折算benchmark性能

图 2 单卡 128 GB 放不下什么



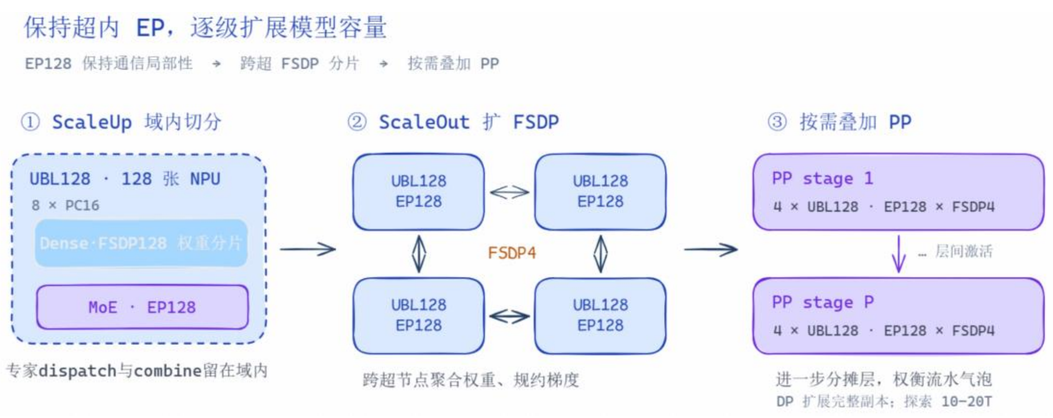
每驻留专家 0.7004 GiB；非专家权重 12.10 GiB 为实测，不可再拆

根据冷热画像分别卸载至主机 DDR

Engram 走随机 gather，路由专家由 kt-kernel 在 CPU 上计算，通过 device 与 host 并行执行和热专家预取，降低 TPOT 延时。

prompt token	输出 token	TTFT	TPOT
1,024	256	19.78 s	22.99 ms
4,096	128	21.40 s	22.57 ms

Deepseek V4.1 昇腾实践：预训练与RL

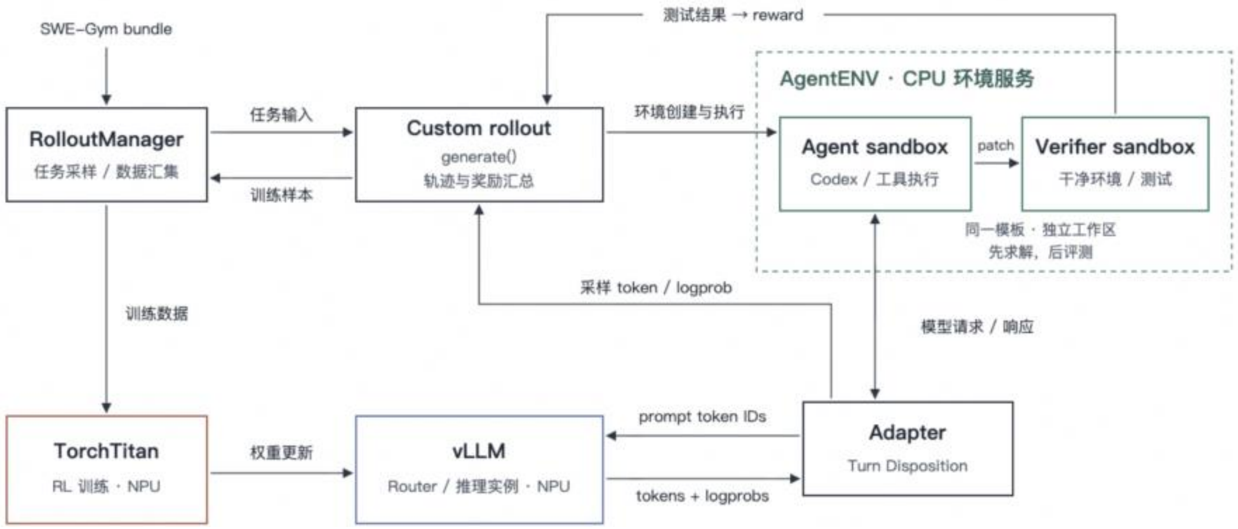


保持超内 EP，逐级扩展模型容量
EP128 保持通信局部性 → 跨超 FSDP 分片 → 按需叠加 PP
① ScaleUp 域内切分
UBL128 · 128 张 NPU
8 × PC16
Dense·FSDP128 权重分片
MoE · EP128
专家dispatch与combine留在域内
② ScaleOut 扩 FSDP
UBL128
EP128
UBL128
EP128
FSDP4
UBL128
EP128
UBL128
EP128
跨超节点聚合权重、规约梯度
③ 按需叠加 PP
PP stage 1
4 × UBL128 · EP128 × FSDP4
… 层间激活
PP stage P
4 × UBL128 · EP128 × FSDP4
进一步分摊层，权衡流水气泡
DP 扩展完整副本：探索 10-20T

PC16 集群 - 预训练 DeepSeek-V4-Flash

基于 TorchTitan-NPU，在 PC16 集群上完成 DeepSeek-V4-Flash预训练配置的端到端验证，使能通过 FSDP、EP 与 CP 并行切分、mxFP8 量化、权重预取和通信域分离。

配置	内存占用（GB）	吞吐（tokens/s/卡）	MFU
128P · 1M	77.95	**1441**	75.17%
64P · 4K	103.95	**2131**	31.69%



SWE-Gym bundle
测试结果 → reward
AgentENV · CPU 环境服务
RolloutManager
任务采样 / 数据汇集
任务输入
训练样本
Custom rollout
generate()
轨迹与奖励汇总
环境创建与执行
Agent sandbox
Codex / 工具执行
patch
Verifier sandbox
干净环境 / 测试
同一模板 · 独立工作区
先求解，后评测
采样 token / logprob
训练数据
模型请求 / 响应
TorchTitan
RL 训练 · NPU
权重更新
vLLM
Router / 推理实例 · NPU
prompt token IDs
tokens + logprobs
Adapter
Turn Disposition

鲲鹏/昇腾950/灵衢协同AgenticRL实践

- 对接鲲鹏AgentENV，支持基于多种Harness 的AgenticRL训练和SWE任务的稳定Reward收敛。
- 完成 DSV4 Flash 在 950DT 上的训推全链路 bitwise 对齐，在权重同步场景下，通过算法、框架到CANN算子Kernel的层层校准，实现训推 logprob MAE 为 0.0。
- 支持 MXFP8 和 MXFP4 的 RL 低精度训练流程，加速 Rollout 推理，并优化训推权重传输代价与静态内存占用。
- 支持 M 级长序列训推，训练和推理均使用 CP 并行，多卡分担 activation 峰值内存。
- 依托灵衢UB高速互联能力，实现 EP/CP/FSDP 高效通信及训推权重快速同步，提升大规模 RL 训练效率。

开源实践与相关参考

主标题	链接
TileLang 算子Agent开发和优化实践	https://gitcode.com/cann/cannbot-skills/tree/master/plugins-official/tilelang-op-orchestrator
基于Agent-Friendly算子编程范式CANNBot-DSL的高性能融合算子开发实践	https://gitcode.com/cann/cannbot-dsl
通信编程体系：基于UB memory和URMA通信编程接口的通算并行算子开发实践	https://gitcode.com/cann/cann-recipes-infer/tree/master/docs/models/deepseek_v4_1/deepseek_v4.1_asc_comm_tech_report.md
DeepSeek-V4.1-Flash推理优化实践	https://gitcode.com/cann/cann-recipes-train/blob/master/docs/llm_pretrain/ascend950_superpod_moe_pretrain_system_reference.md
DeepSeek-V4.1-Flash基于vllm推理部署实践	https://gitcode.com/cann/cann-recipes-infer/tree/master/docs/models/deepseek_v4_1/deepseek_v4.1_flash_cann_tech_report.md https://github.com/kvcache-ai/Mooncake
DeepSeek-V4.1-Flash单机部署低时延推理优化实践	https://gitcode.com/cann/cann-recipes-infer/tree/master/docs/models/deepseek_v4_1/deepseek_v4.1_low_latency_tp_guide.md
DeepSeek-V4.1-Flash 单卡部署推理优化实践	https://gitcode.com/cann/cann-recipes- infer /tree/master/docs/models/deepseek_v4_1/dsv41-flash-950-single-card.md
基于PyTorch原生框架的DeepSeek-V4.1-Flash模型预训练实践	https://gitcode.com/cann/cann-recipes-train/blob/master/docs/llm_pretrain/ascend950_superpod_moe_pretrain_system_reference.md
基于鲲鹏CPU、昇腾950、灵衢协同的Agentic RL训练实践	https://gitcode.com/cann/cann-recipes-train/tree/master/docs/llm_rl/kunpeng_ascend_agentic_rl_infrastructure.md
鲲鹏CPU对AgentENV的适配与优化实践	https://gitcode.com/KunpengSDK/agentENV/blob/main/agentenv-kunpeng-optimization-practice.md
DeepSeek-V4.1-Flash模型低精度量化训练实践	https://gitcode.com/cann/cann-recipes-train/tree/master/docs/llm_pretrain/ascend950/low_precision_training.md
DeepSeek-V4 DSpark草稿模型昇腾训练实践	https://gitcode.com/cann/cann-recipes-train/tree/master/llm_sft/DeepSpec
DeepSeek-V4 单机LoRA 微调实践	https://gitcode.com/cann/cann-recipes-train/tree/master/docs/feature_guides/deepseek_v4_lora.md
推理模型优化Agent 部署调优实践	https://gitcode.com/cann/cann-recipes-infer/blob/main/docs/agent/DeepSeekV4.1-Flash-Model-Agent.md https://gitcode.com/cann/cannbot-knowledge
AMCT 低精度量化Agent量化实践	https://gitcode.com/cann/amct/tree/master/examples/models/deepseekv4.1/DeepSeekV4.1-Flash-Quantization-Agent.md

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态
社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem
Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann